⚠ **TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE**

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the document register for the full explanation.

Status: Partial — see Gaps. Clause: 5.8 — Software Release · **Register:** Document Register

What the standard requires (Class C — every sub-clause of 5.8)

REF	TITLE	APPLIES AT	OUTPUT
5.8.1	Ensure software verification is complete	A, B, C	—
5.8.2	Document known residual anomalies	A, B, C	All known residual anomalies, documented
5.8.3	Evaluate known residual anomalies	**B, C**	Evaluation confirming residual anomalies do not contribute to unacceptable risk
5.8.4	Document released versions	A, B, C	The version of the medical device software being released, documented
5.8.5	Document how released software was created	**B, C**	The procedure and environment used to create the released software, documented
5.8.6	Ensure activities and tasks are complete	**B, C**	Evidence all plan activities/tasks are complete, with associated documentation

REF	TITLE	APPLIES AT	OUTPUT
5.8.7	Archive software	A, B, C	Archive of the software, configuration items and documentation, retained for the longer of software lifetime or regulatory retention
5.8.8	Assure reliable delivery	A, B, C	Procedures for replication, media labelling, packaging, protection, storage and delivery

A distinction worth making before anything else

The version chip visible on every page (see `version` test group) records **which edition of the standard the course content covers** — `Edition 1 · 2006+A1:2015` . That is a content fact, verified by REQ-01 in 03. It is **not** the software's own release version, which is what 5.8.4 actually asks for. Conflating the two would misrepresent both — this document is about the latter, which, as the Gaps section below states, does not currently exist.

5.8.1–5.8.2 — Verification complete, anomalies documented

08 — System Testing is the verification-complete gate: a release is only considered ready when `tests/test_site.py` 's full run passes. Known residual anomalies are exactly what `tests/anomaly_log.csv` tracks — any row still `Open` at release time is, by definition, a known residual anomaly, with its own `ANOM-####` id, first-seen date, and detail.

5.8.3 — Evaluating residual anomalies

This is the one step the anomaly log does not do automatically, by design — 13 — Problem Resolution explains why a machine should record that an anomaly exists but a person should judge whether it's acceptable to release with it open, not the other way around. For this project that evaluation currently happens

informally rather than being a documented release-gate step; recorded as a gap below.

5.8.4 — Released versions

No version identifier is currently assigned to a release. See Gaps.

5.8.5 — How the software was created

The environment is fully reproducible without a build step, since the site is unbundled HTML/CSS/JS served as-is (see 02 — Software Development Plan, §5.1.4). Hosting is GitHub Pages, serving directly from the branch — there is no CI pipeline (`.github/workflows` does not exist in this repository) and no separate build artifact distinct from the source tree itself.

5.8.6 — Plan activities complete

Cross-referenced against 02 — Software Development Plan: of the planned activities, unit-level verification (06) and a separate integration test phase (07) are the two not complete as planned — recorded there, not hidden here.

5.8.7–5.8.8 — Archive and delivery

Git *is* the archive: every released state of every file is retrievable by commit, and GitHub Pages' deployment history provides a delivery record. What git does **not** currently provide is a *labelled* release boundary — see Gaps.

Gaps

- **No release versioning scheme.** No git tags exist in this repository (`git tag` returns nothing as of this writing), no `CHANGELOG.md` , and no version string embedded in the software itself. 5.8.4's "document the version of the software being released" has no current answer beyond "the commit at the tip of `main` at deployment time."

- **No formal release gate or checklist.** Deployment to GitHub Pages happens whenever `main` is updated; there is no explicit "release" event distinct from "a commit landed," so 5.8.1's completeness check and 5.8.6's activity-completeness check are not currently performed as a discrete, recorded step.
- **Suggested next step, not yet taken:** tag each meaningfully complete state (e.g. `v1.0.0`) and record, at minimum, the anomaly log's open-anomaly count at that commit as the 5.8.2/5.8.3 evidence for that release. This is named as a suggestion rather than implemented here, consistent with 05's reasoning: a process adopted at the next real release is more genuine than one backdated for this document set's sake.